# DESIGN

# CONST-040 Capability-Model Design — HXC-117 / HXC-118 / HXC-119

**Revision:** 1 **Created:** 2026-07-12 **Last modified:** 2026-07-12 **Status:** active **Scope:** design + phased sub-task plan only — NO source changes made in this pass (§11.4.145 independent impact-research agent; read-only per dispatch instructions)

## Table of contents



---

## 1. Current capability model (as-is, cited)

### 1.1 `VerificationResult` / `VerifiedModel` — the only two structs CONST-040 could gate from

File: `helix_code/internal/verifier/types.go`

- `VerifiedModel` (lines 24-61) carries model-level capability booleans: `SupportsStreaming`, `SupportsTools`, `SupportsFunctions`, `SupportsCode`, `SupportsVision`, `SupportsAudio`, `SupportsVideo`, `SupportsReasoning`, `SupportsEmbeddings` (line 43), `SupportsJSONMode`, plus a free-form `Capabilities []string` (line 59) and `Tags []string` (line 60).
- `VerificationResult` (lines 81-108) — the on-demand verification result — carries a **different, narrower** boolean set: `SupportsToolUse`, `SupportsCodeGeneration`, `SupportsEmbeddings` (line 95), `SupportsStreaming`, `SupportsJSONMode`, `SupportsReasoning`, plus seven `CodeXxx`/`TestGeneration`/… flags (lines 99-105).
- **Nowhere in either struct** is there a field for MCP, LSP, ACP, RAG, Skills, or Plugins. doc . go line 33 lists `CONST-040: MCP/LSP/ACP/Embedding/RAG/`

`Skills/Plugins` as a **package-level constitutional citation**, not an implemented contract — it documents the mandate, not a fulfilled one.

## 1.2 The only CONST-040 consumer that exists today

File: `helix_code/internal/llm/ensemble_resolver.go`

- `ensembleVerifiedModelFor` (lines 61-113) reads `verifierAdapter.GetVerifiedModels(ctx)` and filters on `m.Verified`, `m.Deprecated`, and `m.SupportsEmbeddings` (line 89) to exclude embedding-only models from the ensemble's chat-member candidate pool.
- `verifierCapabilitiesAreEmbeddingOnly` (lines 121-141) matches the free-form `Capabilities []string` tokens against a 3-entry `embeddingCapabilityHints` map (lines 42-46: `"embedding"`, `"embeddings"`, `"embed"`).
- **This is the ONLY place `SupportsEmbeddings` (or any verifier capability field) is read anywhere in the inner app** (confirmed via `grep -rn "SupportsMCP\|SupportsLSP\|SupportsACP\|SupportsRAG\|SupportsSkills\|SupportsPlugins\|SupportsEmbedding" --include="*.go" . | grep -v _test.go` — zero hits outside `verifier/types.go` and `llm/ensemble_resolver.go`).

## 1.3 How MCP / LSP / Skills / Plugins actually decide availability today (verifier-gating: ZERO)

- **MCP** — `helix_code/internal/mcp/registry.go` (`Manager.Start`, lines 104-129; `buildClient`/`buildTransport`, lines 132-137 and 276-303) wires servers purely from a user-authored `mcp.Config` (`ServerSpec` list, transport = stdio/http/sse/ws). No verifier import, no capability check anywhere in the package (confirmed: package-level `grep` for verifier import in `internal/mcp/*.go` returns nothing).
- **LSP** — `helix_code/cmd/cli/lsp_cmd.go` (lines 108-130, `runLSPSubcommand`) queries the live `commands.LSPManager` / `tools.LSPServerSpec` curated allowlist directly (`exec.LookPath` + process management per the file's own doc comment, lines 13-16). No verifier read.
- **Skills** — `helix_code/cmd/cli/skills_cmd.go` reads `commands.SkillRegistry`/`commands.SkillLoader` (filesystem-scanned skill definitions), never the verifier.
- **Plugins** — `helix_code/internal/plugins/activation.go` (`LoadPlugins` lines 16-22, `MaybeRunPlugin` lines 43-68) scans a plugin directory for `manifest.yaml` and dispatches on a `@plugin:<name> <action>` prompt regex. No verifier import anywhere in `internal/plugins/*.go`.

- **RAG** — no package exists in `helix_code/internal/` at all. The only "RAG" hits in the inner app are prose in two model **descriptions** (`internal/llm/local_llm_manager.go:129` "Open-source local AI assistant with RAG capabilities"; `internal/llm/bedrock_provider.go:377,384` Cohere Command-R model blurbs) — cosmetic strings, not a capability or a wired retrieval path.

**Conclusion (HXC-117 root finding):** CONST-040's "MCP/LSP/ACP/Embedding/RAG/Skills/Plugins capability flags MUST be sourced from verifier `VerificationResult`" is **unimplemented for six of seven items**. Only the embeddings exclusion in the ensemble resolver honors the mandate, and even that reads `VerifiedModel.SupportsEmbeddings`, not `VerificationResult` (the two structs are siblings, not the same type — CONST-040 names `VerificationResult` specifically, doc.go line 33).

---

# 2. HXC-117 — capability-flag design

## 2.1 Design goal

Extend `VerificationResult` (and, where the field is genuinely model-scoped rather than session/tool-scoped, `VerifiedModel`) with the five missing capability signals, and wire **at least one real read-point per subsystem** so the flags are not a repeat of the current "declared but never read" bluff.

## 2.2 Field-by-field design

`internal/verifier/types.go` — add to `VerificationResult` (extends the existing bool block at lines 93-98):

```
// CONST-040 capability flags — MCP / LSP / ACP / RAG / Skills /
        Plugins.
// Each reports whether the verified model/provider combination
        has been
// confirmed (by LLMsVerifier, the CONST-036 single source of
        truth) to
// support the corresponding integration surface. False/zero-
        value means
// "not verified as supporting" — NEVER "verified as NOT
        supporting"; a
// verifier that has not run the relevant probe MUST report
        false, and
// callers MUST treat false as "fall back to config-driven
        behavior",
// never as "actively disabled" (no user-facing capability may
        regress
```

```
// silently because the verifier hasn't probed it yet — see §5
        Phase 1
// rollout risk below).
SupportsMCP     bool `json:"supports_mcp"`
SupportsLSP     bool `json:"supports_lsp"`
SupportsACP     bool `json:"supports_acp"`
SupportsRAG     bool `json:"supports_rag"`
SupportsSkills  bool `json:"supports_skills"`
SupportsPlugins bool `json:"supports_plugins"`
```

Why `VerificationResult` and not (only) `VerifiedModel`: `VerificationResult` is what CONST-040's doc.go line 33 literally names, and it is the artifact of an **on-demand** verification run (`Status` field, `CompletedAt` field) — the natural place to attach "this model+provider pairing was probed for tool-use / MCP-bridging / RAG-retrieval fitness." `VerifiedModel` (the catalogue-listing type) should mirror the same six fields for consistency with the existing `SupportsEmbeddings` duplication pattern already present in both structs (types.go lines 43 and 95) — same rationale, same shape, zero new design risk.

### 2.3 Per-subsystem read-points (what must change to make the flags load-bearing, not decorative)

| Subsystem | Current file:line (verifier-blind) | Proposed read-point |
|---|---|---|
| MCP | `internal/mcp/registry.go:104` `Manager.Start` — dials `AlwaysLoad` servers unconditionally from `mcp.Config` | Before `wg.Add(1)`/dial (line 118-125), when a server spec names a `Model` (new optional `ServerSpec.RequiredModel` field, config-only — MCP servers are user-installed tools, not model properties, so the model check is advisory: "does the active ensemble model support MCP tool-calling" — read via the package-level `verifierAdapter` the same way `ensemble_resolver.go:62` does) |
| LSP | `cmd/cli/lsp_cmd.go:108` `runLSPSubcommand` — | LSP servers are language-tooling processes, not model capabilities — `SupportsLSP` |

| Subsystem | Current file:line (verifier-blind) | Proposed read-point |
|---|---|---|
| | delegates straight to `commands.LSPCommand` | is more honestly a **provider/session** capability ("can this model consume LSP diagnostics as tool-call context"); read-point = the tool-loop's context-injection step in `internal/agent/tool_loop.go` (not yet inspected in this pass — Phase 2 task) before attaching LSP diagnostics to a prompt |
| ACP | none exists (HXC-119) | the ACP agent-side session/prompt handler (§4.4) checks `SupportsACP` before advertising ACP `session/new` capability negotiation for a given backing model |
| RAG | none exists (HXC-118) | the RAG pipeline construction step (§3.4) checks `SupportsRAG` (or, more usefully, `SupportsEmbeddings` on the embedding model doing the vector encode — RAG capability is really a property of the **embedding model**, not the generation model; see §3.3 open design question) before offering retrieval-augmented mode |
| Skills | `cmd/cli/skills_cmd.go` reads `commands.SkillRegistry` directly | `internal/agent/skill_dispatcher.go` (not yet inspected — Phase 2) is the more likely load-bearing site: gate skill-invocation-via-LLM-function-call on `SupportsSkills` (i.e., "can this model reliably drive structured skill dispatch") |
| Plugins | | |

| Subsystem | Current file:line (verifier-blind) | Proposed read-point |
|---|---|---|
| | `internal/plugins/activation.go:43 MaybeRunPlugin` | same shape as skills — the regex-dispatch path (line 30, 43) doesn't need a model-capability gate (it's a deterministic string match, not an LLM decision), but a **future** LLM-driven plugin-selection mode (CONST-046-compliant, no hardcoded `@plugin:` syntax requirement) would read `SupportsPlugins` at the point it asks the model "which plugin should handle this prompt" |

**Honest boundary (§11.4.6):** LSP, Skills, and Plugins are today **config/filesystem-driven, not model-driven** capabilities — a model doesn't "support" having an LSP server installed. The CONST-040 flags for these three are best understood as *"is this model verified as capable of consuming/driving the subsystem via tool-calls,"* not *"is the subsystem itself available."* This is a genuine design ambiguity in the original CONST-040 mandate; the sub-task plan (§5) allocates explicit time to resolve it with the operator via the constitution's own §11.4.66 interactive-clarification mechanism before wiring gates that could be mis-scoped.

### 2.4 Blast radius of adding the six fields alone

Adding six `bool` fields to `VerificationResult` (a JSON-tagged struct read from an HTTP client, `client.go`) is additive and non-breaking: existing `json.Unmarshal` calls on older verifier-service responses simply leave the new fields at their zero value (`false`), and no existing code path reads them yet, so **zero behavior change** until a read-point (2.3) is wired. This is the single lowest-risk part of the whole three-item plan and can land first, independently, as its own commit.

---

## 3. HXC-118 — RAG submodule integration plan

### 3.1 Submodule facts (verified by direct build)

- Path: `submodules/rag/` (flat-grouped layout per `.gitmodules`, consistent with CONST-051(C)).

- Module: `digital.vasic.rag`, `go 1.24.0` (compatible with helix_code's go 1.26 — no toolchain conflict).
- `go build ./...` inside `submodules/rag/` **succeeds** (verified this session, exit 0) — the submodule is not broken, it is simply unimported.
- `helix-deps.yaml` declares zero own-org dependencies (genuine leaf submodule, `Catalogue-Check` comment already present per §11.4.74/ CONST-054).
- Public packages: `pkg/retriever` (Document, Options, Retriever interface, MultiRetriever), `pkg/chunker`, `pkg/reranker`, `pkg/hybrid`, `pkg/pipeline` (Builder/Pipeline fluent API — `pipeline.NewPipeline().Retrieve(r).Rerank(reranker).Format(formatter).Execute(ctx, query) (*Result, error)`).
- **Zero inner-app imports today** (confirmed via `grep -rln "digital.vasic.rag" helix_code/` returning nothing) — HXC-118's "0 inner-app imports" finding is confirmed as FACT, not a guess.

## 3.2 What "integrate" concretely means

1. Add `digital.vasic.rag` as a `go.mod` require in `helix_code/go.mod`, with a `replace digital.vasic.rag => ../submodules/rag` directive during development (mirrors the §11.4.76 containers-submodule consumption pattern: `replace` directive during dev, pinned commit SHA in production per that anchor's clause 2).
2. Implement the **only missing piece** the submodule doesn't provide: a concrete `retriever.Retriever` backed by HelixCode's own storage. The submodule ships the *interfaces and pipeline machinery* (retriever.go's `Retriever` interface, line 37-40) but deliberately no concrete vector-store-backed implementation (it is a decoupled, project-not-aware submodule per §11.4.28(B) — it cannot know HelixCode's embedding provider or persistence layer). This concrete adapter is genuinely new HelixCode-side code, NOT a submodule extension (the submodule's contract is correctly abstract; HelixCode supplies the concrete backing per the standard adapter pattern already used for `verifierAdapter` in `internal/llm`).
3. Candidate concrete-retriever backing: `internal/memory` (not yet inspected this pass — the project already has a memory/persistence subsystem per the repo-layout table in CLAUDE.md §3.2.1) or `internal/persistence` combined with an embedding call through the existing `llm.Provider` interface (any provider whose `VerifiedModel.SupportsEmbeddings` is true — this is the natural intersection with HXC-117 §2.3's RAG read-point).

## 3.3 Open design question (flagged honestly, not guessed — §11.4.6)

Which embedding provider backs the vector encode step is a genuine open decision requiring either (a) an existing `internal/llm` embedding call path

(UNCONFIRMED whether one exists — not located in this pass; grep for embedding outside verifier/ensemble_resolver returned only description strings, §1.2/§3.1) or (b) net-new embedding-provider wiring. This is flagged as a Phase-1 investigation sub-task (§5), not resolved here — declaring an unverified embedding path would itself be a bluff.

## 3.4 Wiring points (capability flag + request-flow hook)

- **Capability flag**: a new `RAGEnabled bool` (or reuse of `SupportsRAG` from §2.2) on the session/request config, defaulted false, surfaced via a new CLI flag (`--rag`) and/or a `commands` slash-command (`/rag on|off|status`) following the exact `mcp_cmd.go`/`lsp_cmd.go` cobra-subcommand pattern already established (§1.3) — this keeps HXC-118 consistent with the existing CLI-surface conventions instead of inventing a new one.
- **Request-flow hook**: the natural insertion point is immediately before the prompt is sent to the LLM provider inside `cmd/cli/main.go:handleGenerate` (line 1714 — the BLUFF-001-anchored real-generation path) and `cmd/cli/main.go:handleInteractive` (line 2298 — the REPL loop): when RAG is enabled, run `pipeline.Execute(ctx, userPrompt)` first, and prepend/inject `Result.Output` (formatted retrieved context) into the prompt sent to `provider.Generate`/`GenerateStream`. This is the same shape as the existing MCP tool-injection and LSP-diagnostic-injection points (both of which live in the same handler family) — no new architectural pattern, just a new injection stage.
- **New package**: `internal/rag/` (adapter package, analogous to how `internal/mcp` wraps the MCP protocol) housing the concrete retriever + a thin `Manager` that owns pipeline construction, config load/save (mirroring `mcp.LoadConfig`/`mcp.SaveConfig`), and the CONST-046-compliant (no hardcoded strings) status/label surface.

## 3.5 Catalogue-check citation (§11.4.74)

```
Catalogue-Check: reuse vasic-digital/rag (submodules/rag,
digital.vasic.rag) — 100% match for the retrieval/rerank/pipeline
abstraction layer; extend NOT required at the submodule level (it
correctly stays project-not-aware); the missing 20% (concrete
storage-backed retriever) is HelixCode-side adapter code by
design (§11.4.28(B) decoupling), not a submodule gap.
```

---

# 4. HXC-119 — ACP (Agent Client Protocol)

## 4.1 What ACP is (cited, §11.4.8/§11.4.99 deep research performed this session)

The Agent Client Protocol is a JSON-RPC 2.0-based protocol, modeled explicitly on the Language Server Protocol, that standardizes communication between code editors/IDEs ("clients") and AI coding agents ("agents") — eliminating N×M custom-integration overhead the same way LSP did for language tooling. Local agents run as editor sub-processes communicating over stdio; remote/HTTP-WebSocket transport is noted by the official docs as work-in-progress. It was introduced by Zed in August 2025 and JetBrains joined shortly after, turning it into a multi-editor interoperability layer (JetBrains now ships first-class ACP support per jetbrains.com/acp). The protocol defines a session lifecycle (`initialize` → `session/new` → `session/prompt` → streaming `session/update` notifications) plus auxiliary methods for filesystem access (`fs/read_text_file`), terminal execution, and permission requests — the same shape HelixCode's own `internal/mcp` package already implements for MCP (session-based JSON-RPC over stdio/HTTP/SSE/WS, `internal/mcp/doc.go` lines 1-199).

## 4.2 Is there a reference implementation to build against? YES — including Go

- Official spec + docs: `agentclientprotocol.com` (maintained jointly by Zed + JetBrains, per `github.com/agentclientprotocol/agent-client-protocol`).
- **Official-adjacent Go SDK**: `github.com/coder/acp-go-sdk` (pkg.go.dev confirms it as a maintained Go package: "acp package - github.com/coder/acp-go-sdk"). It provides:
  - `Agent` interface (what HelixCode would implement — HelixCode IS the coding-assistance agent in ACP terms, analogous to how Claude Code / Gemini CLI are ACP agents editors connect to) and `Client` interface (the editor side — not HelixCode's role).
  - Connection constructors `acp.NewAgentSideConnection(agent, stdout, stdin)` for the agent role.
  - Typed request/response plumbing for `initialize`, `session/new`, `session/prompt`, `session/update`, `fs/read_text_file`, `terminal`, `permission`.
  - Content-block helpers (`TextBlock`, `ImageBlock`, `AudioBlock`, `ResourceBlock`) and an extension mechanism for vendor-specific underscore-prefixed methods.

- Additional independent Go implementations exist (`ironpark/acp-go`, `rokku-c/acp-go`) — corroborating the protocol is genuinely Go-implementable, not merely TypeScript/Python-only.

## 4.3 Finding: ACP is IMPLEMENTABLE, NOT structurally impossible

Per §11.4.112, a `Won't-fix: structurally-impossible` classification requires PROVEN platform/hardware/protocol impossibility. The opposite is proven here: an official-adjacent, actively maintained Go SDK exists (`github.com/coder/acp-go-sdk`), the protocol is JSON-RPC-over-stdio (a transport HelixCode's own `internal/mcp/transport_stdio.go` already implements for the structurally-analogous MCP protocol), and HelixCode already has the exact architectural role required — a CLI program with a turn-based prompt/response loop (`cmd/cli/main.go`: `handleGenerate` line 1714, `handleInteractive` line 2298) — needed to be the ACP **Agent** side. **Deciding evidence for "implementable": the existence and current maintenance of `github.com/coder/acp-go-sdk` (verified via pkg.go.dev + GitHub search this session, access date 2026-07-12) providing a ready-made `Agent` interface + stdio connection constructor that maps directly onto HelixCode's existing CLI turn loop.**

## 4.4 Implementation approach (design sketch)

1. **Dependency**: add `github.com/coder/acp-go-sdk` to `helix_code/go.mod` (pure Go module dependency, no submodule/vendoring needed — this is a published library per §11.4.74's "consumed as published package" pattern, same class as CodeGraph's npm consumption).
2. **New package** `internal/acp/` (parallel to `internal/mcp/`): implements `acp.Agent` by wrapping the existing CLI turn-handling logic — `Initialize` reports HelixCode's capabilities (tool-use, streaming, etc., itself potentially informed by the verifier per HXC-117 §2.3's ACP read-point); `NewSession` creates a `session.Session` (existing `internal/session` package, `session.go`); `Prompt` delegates to the same code path `handleGenerate` (main.go:1714) already exercises against real providers (BLUFF-001-clean — no simulation risk since it reuses the existing real-provider call).
3. **New CLI entrypoint**: `helixcode acp` subcommand (cobra, following the exact `lsp_cmd.go`/`skills_cmd.go`/`mcp_cmd.go` pattern at `cmd/cli/acp_cmd.go`) that runs `acp.NewAgentSideConnection(agentImpl, os.Stdout, os.Stdin)` and blocks — this is the stdio sub-process mode an editor (Zed, JetBrains) would launch HelixCode in, exactly mirroring how `internal/mcp/transport_stdio.go` already spawns/talks-to MCP servers from the other side of an analogous protocol.
4. **Streaming updates**: `session/update` notifications map onto the existing `GenerateStream` real-time token path already used for `--stream` in

`handleGenerate` (no new streaming plumbing needed, only a new notification-emission wrapper).

5. **Permission requests**: map onto HelixCode's existing `internal/approval/` `internal/tools/confirmation`/`internal/tools/permissions` packages (already present per the go.mod-adjacent directory listing in main.go's import block, lines 27, 51-54) — ACP's `permission` method is functionally the same concept HelixCode already implements for tool-call approval, so this is a protocol-translation layer, not new policy logic.

### 4.5 Risk / honest gaps

- Remote (HTTP/WebSocket) ACP transport is explicitly WIP in the official spec (per agentclientprotocol.com/get-started/introduction, fetched this session) — Phase 1 should scope to stdio-only, matching the officially-stable transport.
- `coder/acp-go-sdk` is a third-party (not Zed/JetBrains-authored) SDK; its version/stability should be pinned and its own test coverage spot-checked before adoption (Phase 1 sub-task, §5).
- HXC-119 is **NOT** a §11.4.112 structural-impossibility candidate — closing it that way would itself be a §11.4.150 violation (closing/structural-verdicting without the mandatory deep multi-angle research pass, which this section IS, cited below).

---

## 5. Phased sub-task breakdown + blast-radius/risk (§11.4.145)

All three items share one root cause (an unfulfilled/incomplete capability-model contract) and interlock (RAG's capability flag and ACP's capability flag both live in the same `VerificationResult` struct HXC-117 extends) — hence they were dispatched to one design pass instead of three independent ones, per the task's own framing.

### HXC-117 — capability flags

| Phase | Sub-task | Blast radius | Risk |
|---|---|---|---|
| 1 | Add six `bool` fields to `VerificationResult` + mirror on `VerifiedModel` (types.go) | Additive struct fields only; zero read-points wired yet | **Low** — no behavior change (§2.4); safe to land standalone |
| 2 | Resolve the MCP/LSP/Skills/Plugins "model-capability vs subsystem- | none (research/decision only) | **Low** — pure decision-gathering; wrong |

| Phase | Sub-task | Blast radius | Risk |
|---|---|---|---|
| | availability” ambiguity (§2.3 honest boundary) via §11.4.66 interactive clarification with the operator BEFORE wiring any gate | | assumption here would misdirect Phase 3 |
| 3 | Wire the MCP read-point (`internal/mcp/registry.go` `Manager.Start`) | Touches MCP server-dial path; regression risk = a verifier-unreachable state must fail OPEN to current unconditional-dial behavior, never fail closed and silently stop dialing configured servers | **Medium** — MCP is a load-bearing developer-tool surface; requires §11.4.145 angle (2) regression call-graph check on every `Manager.Start` caller before merge |
| 4 | Wire the LSP/Skills/Plugins read-points (locations TBD pending Phase 1 investigation of `tool_loop.go`/`skill_dispatcher.go`) | Same shape as Phase 3 | **Medium**, same fail-open discipline |
| 5 | Full four-layer test coverage (§11.4.4(b)) + HelixQA bank + regression guard registration (§11.4.135) per wired subsystem | Test-only | **Low** |

## HXC-118 — RAG integration

| Phase | Sub-task | Blast radius | Risk |
|---|---|---|---|
| 1 | Investigate `internal/memory/internal/persistence` + confirm/deny existence of an embedding-call | Read-only investigation | **Low** |

| Phase | Sub-task | Blast radius | Risk |
|---|---|---|---|
| | path in `internal/llm` (§3.3 open question) — MUST complete before any adapter code is written | | |
| 2 | Add `go.mod` require + `replace` directive for `digital.vasic.rag`; confirm `go build ./...` still succeeds inner-app-wide | Dependency-graph only, no runtime wiring | **Low** — mirrors §11.4.76 pattern already proven safe for `containers` |
| 3 | Implement concrete `retriever.Retriever` adapter in new `internal/rag/` package, backed by the Phase-1-confirmed storage + embedding path | New package, no existing call-sites touched | **Low-Medium** — new code, but additive (nothing currently calls it) |
| 4 | Wire the CLI surface (`cmd/cli/acp_cmd.go`-style `rag_cmd.go`) + the `handleGenerate`/`handleInteractive` injection hook (§3.4), default-OFF | Touches the two most heavily-used CLI handlers (main.go:1714, 2298) | **Medium-High** — these are the BLUFF-001-anchored real-generation paths; any change here MUST preserve the existing real-provider-call contract exactly when RAG is disabled (default), and the §11.4.145 angle (2) regression check must enumerate every existing caller/test of |

| Phase | Sub-task | Blast radius | Risk |
|---|---|---|---|
| | | | both handlers before merge |
| 5 | Full four-layer coverage + HelixQA + regression guard | Test-only | **Low** |

### HXC-119 — ACP

| Phase | Sub-task | Blast radius | Risk |
|---|---|---|---|
| 1 | Pin + vendor-audit `github.com/coder/acp-go-sdk` (version, license, its own test coverage) | Dependency-graph only | **Low** |
| 2 | Implement `internal/acp/acp.Agent` wrapper delegating to the existing turn-handling logic (§4.4.2) | New package | **Low-Medium** |
| 3 | New `cmd/cli/acp_cmd.go` stdio entrypoint (`helixcode acp`) | New cobra subcommand, additive — does not touch existing subcommand routing beyond one new `root.AddCommand` call, same shape as every other `newXxxCmd` registration already in `main.go` | **Low** — purely additive surface |
| 4 | Wire `session/update` onto the | Touches the streaming call | **Medium** — must confirm (Phase-1- |

| Phase | Sub-task | Blast radius | Risk |
|---|---|---|---|
| | existing `GenerateStream` path (§4.4.4) | inside `handleGenerate`, but only when invoked FROM the new ACP entrypoint (a new caller, not a change to the existing CLI-interactive caller) | of-this-sub-item angle) that `GenerateStream`'s existing callback signature can be reused without modifying its contract for the pre-existing CLI streaming caller |
| 5 | Map ACP `permission` requests onto existing `internal/approval/` `internal/tools/permissions` (§4.4.5) | Touches an existing, security-relevant subsystem | **Medium-High** — permission-request translation is exactly the class of change §11.4.145 angle (5) security + angle (6) host/data safety must independently research before merge; no shortcut here regardless of how mechanical the mapping looks |
| 6 | Full four-layer coverage + a real Zed-or-JetBrains-driven (or `coder/acp-go-sdk`'s own example client) E2E Challenge (§11.4.143-class real-journey discipline — no synthetic-JSON-RPC-only test as the sole proof) + HelixQA + regression guard | Test-only, but MUST include a genuine external-client round-trip per the anti-bluff mandate — a self-talking-to-itself JSON-RPC unit test alone would be a metadata-only bluff | **Low execution risk, HIGH bluff risk if skipped** — flagged explicitly for the implementing agent |

### Cross-cutting risk shared by all three items

Because HXC-117's `VerificationResult` extension is a prerequisite read-point for both HXC-118's and HXC-119's optional capability-gating (§3.4, §4.4.2), **HXC-117 Phase 1 (struct fields) should land first**, independently, before either HXC-118 or HXC-119 Phase 2+ begins — this avoids the three items conflicting on the same struct in parallel work-streams (§11.4.176 exactly-once claim discipline: the `types.go` file-scope should be claimed by exactly one track at a time).

---

# Sources verified 2026-07-12:

- https://agentclientprotocol.com/get-started/introduction
- https://github.com/agentclientprotocol/agent-client-protocol
- https://github.com/agentclientprotocol
- https://www.jetbrains.com/acp/
- https://blog.marcnuri.com/agent-client-protocol-acp-introduction
- https://github.com/coder/acp-go-sdk
- https://pkg.go.dev/github.com/coder/acp-go-sdk
- https://github.com/ironpark/acp-go
- https://pkg.go.dev/github.com/rokku-c/acp-go